Project Checklist

Submission status for the Developer Akademie DevSecOps project *Eigener Minecraft-Server*.

The requirement texts are quoted from the original German checklist so that they stay comparable with the handout. The notes underneath each group are in English, the documentation language of the course, and point to the place in this repository where the requirement is met.

Legend: `[x]` done · `[ ]` open

---

1. Repository

Vorhandene Dateien

- ☑ Es wurde eine `.gitignore` Datei angelegt, die alle irrelevanten Inhalte aus dem git repository ignoriert
- ☑ Es gibt eine `docker-compose.yaml`, die den Anforderungen des nächsten Punkts genügt
- ☑ Eine Datei namens `README.md` ist vorhanden und entsprechend der Kriterien unten erstellt worden
- ☑ Es befinden sich keine weiteren Dateien im Repository, ohne dass diese explizit in der `README.md` benannt und beschrieben werden

Notes. `.gitignore` excludes the real `.env`, Python virtual environments, archives, credential files and any `data/` directory; the server's own runtime data lives in the named volume `mc-data`, outside the project directory. Every tracked file is listed and described in the *Repository Contents* table of `README.md`, including this checklist and its PDF export.

Dockerfile

- ☑ Es gibt ein Dockerfile in dem ein passendes Image zusammengestellt wird, um einen minecraft-server zu starten
  - ☑ siehe **3. Hinweise** für den Download-Link zur Server-Anwendung
- ☑ Im Dockerfile sollen alle notwendigen pakete im Basis-Image installiert und ggf. konfiguriert werden
- ☑ Stelle sicher, dass dein Server immer starten kann, für Umgebungs-Variablen solltest du evtl. Default-Werte verwenden
- ☑ Du solltest dabei kein vorgefertigtes Minecraft Image verwenden

Notes. The build has two stages. Stage one installs `curl` and `jq`, reads the official Mojang version metadata that `minecraft.net/download/server` resolves to, downloads `server.jar` and verifies it against the published SHA1 checksum; a mismatch fails the build. The same stage adds the Fabric mod loader and the mods pinned in `mods.txt`, each verified against its own published checksum. Stage two starts from `debian:trixie-slim` and installs the headless Java 25 runtime that Minecraft 26.3 requires, so all necessary packages are installed into a plain base image rather than inherited from a ready-made Minecraft image. `curl` and `jq` are absent from the shipped image.

Every runtime setting has a default in the `ENV` block of the `Dockerfile`, and `entrypoint.sh` repeats the same defaults as a fallback. One default is deliberately not a working value: `EULA` is `false`, and `entrypoint.sh` stops with an explanatory message until the operator sets it to `true`, because accepting the agreement is not a decision the image may make on their behalf. Verified by starting the image with `EULA=true` as the only variable, no `.env` file and no volume at all: the server reached `Done` after 12 seconds.

docker-compose.yaml

- ☑ Es gibt einen Service, der definiert und konfiguriert wird: `mc-server`
- ☑ Es gibt eine Env Konfiguration für den Minecraft-Server Service bei der alle unkritischen Variablen konfiguriert werden (keine Auth!)
- ☑ Es gibt notwendige Port-Freigaben, sodass der Container aus dem Internet erreichbar ist
- ☑ Es gibt Volumes-Konfigurationen für die Container Daten, sodass die Inhalte auf einem Dateisystem persistiert werden und der Spielstand nicht verloren geht

Notes. The single service is named `mc-server`. Its `environment` block holds only non-critical values and contains no password, token, user name or IP address. The configuration that must not be tracked arrives through the `.env` file, declared with `required: false`: `EULA`, because accepting the agreement is the operator's own decision, and `RCON_PASSWORD`. Host port `8888` is published onto the container port `25565`. The named volume `mc-data` is mounted at `/data` and holds the world, `server.properties`, `ops.json`, `whitelist.json` and the logs.

README.md

- ☑ Die README sollte ein Inhaltsverzeichnis a.k.a. eine Table-of-Contents (ToC) enthalten
- ☑ Eine Sektion mit einer Beschreibung des Repositories muss vorhanden sein. In dieser Beschreibung sollte genannt werden was die wesentlichen Inhalte sind, was der Zweck des Repositories ist
- ☑ Eine Sektion "Quickstart" sollte als Teil der README enthalten sein. Hier sollten kurz Voraussetzungen genannt und eine Schnellstart-Anleitung beschrieben sein
- ☑ Es ausführliche Variante der vorgenannten Sektion so als "Usage" enthalten sein. Hier soll genauer auf die Konfiguration und Konfigurierbarkeit eingegangen werden, d.h. es soll auch erklärt werden wie relevante Passagen modifiziert werden können, um andere Resultate zu erzielen

Notes. The table of contents was checked programmatically against the real headings. *Description* names the essential contents and the purpose. *Quickstart* lists the prerequisites and the commands to get a server running. *Usage* covers the configuration layers, every environment variable, secrets through `.env`, changing the port and the Minecraft version, persistence, console access and the restart behaviour.

---

2. Dokumentation

> Die Dokumentation des Codes, sowie des Projektes soll im Repository in Form einer README Datei stattfinden. Die Dokumentationssprache für alle Projekte (und zugehörige Unterlagen) ist englisch.

Notes. Both are met. `README.md` is the single place of documentation and is written in English, as are all comments in the `Dockerfile`, `entrypoint.sh`, `docker-compose.yaml`, `.env.example` and `check_status.py`.

---

3. Hinweise

The server binary is taken from the Java Edition download that https://www.minecraft.net/en-us/download/server points to. The build resolves it through Mojang's official version metadata rather than a hard-coded link, so the download URL and its SHA1 checksum always belong to the requested version. The Fabric mod loader runs on top of that file instead of replacing it, so the server application itself still comes from the official download.

Allgemeine Hinweise

- ☐ Zusätzlich zu deinem GitHub Repository solltest du ein kurzes Loom Video (maximal 5min.) aufnehmen und bereitstellen, indem du kurz deine Abgabe zeigst und vorstellst was du getan hast

Note. Still open; the recording is made by the author and the link is added to the submission.

Sicherheitshinweise

- ☑ Speichere keine SSH-Keys im Workspace deines Git-Repositories
- ☑ Speichere keine Passwörter, Tokens, oder Benutzernamen in deinem Code. Verwende hierfür stattdessen Environment-Variablen
- ☑ Speichere keine IP-Adressen, oder sonstigen sensiblen Informationen in einem Git Repository

Notes. Verified with pattern searches over all tracked files: no private key blocks, no assignment of a password, token or API key to a literal value, and no IPv4 address anywhere. `.gitignore` additionally excludes `*.pem`, `*.key`, `id_rsa*` and `*.ppk`. The documentation uses the placeholders `<SERVER_IP>`, `<VM_IP>`, `<YOUR_MINECRAFT_USERNAME>`, `<REPOSITORY_URL>` and `<VERSION>`. The RCON password exists only in the untracked `.env` on the deployment host; `.env.example` carries an empty placeholder.

Code-Konventionen

- ☑ Für build-args, environment Variablen und Shell-Variablen gilt folgende Namenskonvention: `UPPER_CASE_WITH_UNDERSCORE`
- ☑ Bei einer Referenz auf eine Variable sollte immer die `{}`-Notation verwendet werden um Fehler in der Interpretation zu vermeiden: `${SOME_VAR_VALUE}`, statt: `$SOME_VAR_VALUE`
- ☑ Es sollten für build-args, oder Environment Variablen "Default"-Werte konfiguriert werden, allerdings nur dann wenn dies Sinn ergibt
- ☑ Kritische Konfiguration wie Tokens, Passwörter oder ähnliches sollte nicht im Code-Repository gespeichert sein, sondern bspw. durch die Verwendung eines `.env`-files in einen Container hineingegeben werden

Notes. All build arguments, environment variables and shell variables use `UPPER_CASE_WITH_UNDERSCORE`, including the locals inside `entrypoint.sh`. A search for a bare `$VAR` reference across the `Dockerfile`,

`entrypoint.sh` and `docker-compose.yaml` returns nothing; every reference is written `${VAR}`. The `jq` filters read the version through `env.MINECRAFT_VERSION` instead of a jq variable, which keeps the convention intact inside the filter as well.

Defaults are set where they make sense: every runtime setting has one, while `LEVEL_SEED` and `RCON_PASSWORD` default to empty on purpose, because an empty seed means "generate randomly" and an empty password keeps RCON switched off.

Testing

- ☑ Der Minecraft-Server ist erreichbar unter der IP-Adresse deiner Cloud-VM auf Port 8888
- ☑ Für das Testen hast du grundsätzlich zwei Wege: das Spiel starten und versuchen dich auf deinen Server zu verbinden, oder mittels eines Skriptes versuchen eine Verbindung zum Minecraft Server aufzubauen. Dazu kannst du das folgende Python Modul verwenden: https://github.com/py-mine/mcstatus
- ☐ (optional) Du kannst dich von deinem Java-Minecraft Client auf deinem Computer mit dem Server auf deiner Cloud-VM verbinden und Minecraft spielen
- ☑ Nach einem Neustart des Servers sind die konfigurierten Daten noch vorhanden und werden nicht gelöscht oder überschrieben. Dazu gehört unter anderem der Spielstand und die Konfiguration des Game-Servers
- ☑ Die Container der Services werden neugestartet, sobald ein Fehler passiert, der zum Terminieren des Containers führt

Evidence

The server runs on a Hetzner Cloud VM (Ubuntu 24.04.5 LTS, Docker 29.9.2, Docker Compose v5.6.1). All checks below were executed against that deployment.

Check	Method	Result
Reachable on port 8888	`check_status.py <VM_IP>` run from a different machine over the internet	answered: version 26.3, 0/10 players, 14 ms latency
Script-based connection	`check_status.py`, which performs a Server List Ping through `mcstatus`	exit code `0`
World survives a restart	marker file placed in the world, then `docker compose down` followed by `up`	marker present, and `No existing world data` appears `0` times, so the world was reused rather than regenerated
Configuration survives a restart	`allow-flight=true` set by hand in `server.properties`, then container recreated	key still `true`; keys managed by environment variables are rewritten from the environment by design, which is documented in the README
Restart after a failure	server application moved away to force a non-zero exit, then `docker restart`	`RestartCount` rose to `7` with `ExitCode=1`; container restored from the unmodified image afterwards
Restart survives a host reboot	`systemctl is-enabled docker` together with `restart: unless-stopped`	`enabled`, so the container comes back after a reboot of the VM
Server starts from its defaults	image started with `EULA=true` as the only variable, no `.env` and no volume	reached `Done` after 12 seconds
Process is unprivileged	`grep ^Uid: /proc/1/status` inside the container	`999`, the `minecraft` account, not `0`
Server application is authentic	SHA1 comparison during the build	`server.jar: OK`
Mods are authentic	SHA512 comparison during the build	one `OK` per entry in `mods.txt`
Mods are loaded	the Fabric loader's mod list in the container log	`Loading 46 mods`, among them Fabric API, Lithium, FerriteCore and spark; Fabric API alone ships as many small modules
The profiler answers	`spark tps` typed into the server console	`20.0` tick rate together with tick durations and CPU usage
Clean shutdown saves the world	modification time of `world/level.dat` before and after `docker compose stop`	rewritten at the moment of the stop; the final log lines are cut off by the TTY while the save itself completes

The optional item is left unchecked because joining with the game client is shown in the video rather than asserted here. The server itself is reachable, as the first row documents.

Extras

The handout asks for extras to be named briefly: the server application is verified against Mojang's SHA1 checksum at build time, the server process runs unprivileged, and it is started with `exec` so it receives `SIGTERM` as PID 1 and saves the world on shutdown.

The server also runs a chosen set of mods on the Fabric loader: Lithium and FerriteCore lower the CPU and memory it needs, which is what a VM of this size calls for, and spark makes the load measurable from the console with `/spark tps`. Fabric API is their shared library. All four are server-side only and pinned by version and SHA512 checksum in `mods.txt`, so an unmodified client can join and the build stays reproducible.